Introduction to Mutimodal AI for Biomedical Applications

[W4-1: Neural Network – Training (1)]

Jun-Hyuk Kim

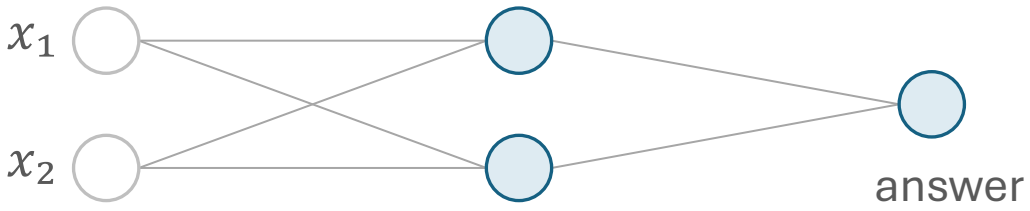
Department of AI

Today’s Journey

1	**What “learning” will mean**	a loop that adjusts numbers
2	**Measuring how wrong we are**	the loss
3	**Making that number smaller**	gradient descent
4	**Four activation functions**	why each one behaves as it does
5	**The wall we run into**	next week’s topic

By the end of today, a network will be choosing its own numbers.

A Network Needs Numbers



each arrow = a **weight**, each neuron = a **bias**

Layer	Weights	Biases
middle (2 neurons)	4	2
output (1 neuron)	2	1

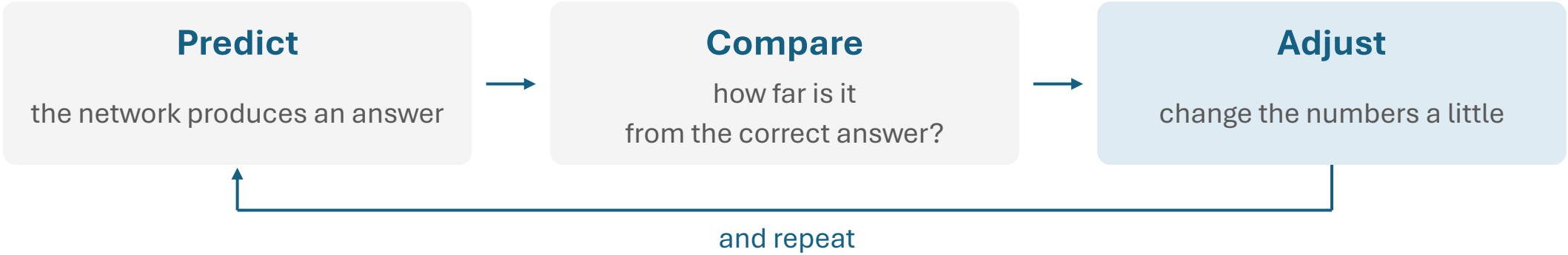
Nine numbers

before the network can do anything

Network	Numbers to choose
this one	9
a small digit reader	about 100,000
a modern language model	billions

Choosing the numbers by hand does not scale, and it never did.

What “Learning” Will Mean Here



Learning adjusting numbers so that the mistakes get smaller.

No understanding, no intention. A loop, repeated.

Learning here means adjusting numbers to reduce mistakes, nothing more.

The Data We Learn From

The training data

x_1	x_2	correct answer y
1.0	2.0	1
0.5	0.5	0
2.0	1.5	1
0.0	1.0	0

inputs — the label

one row = one **training example**

Two different numbers

y the correct answer, from the data

$\hat{y}$ what the network said

Everything today is about the difference between these two.

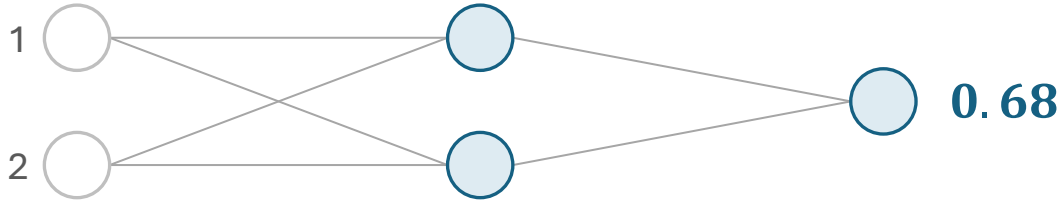
We learn from examples where the answer is already known.

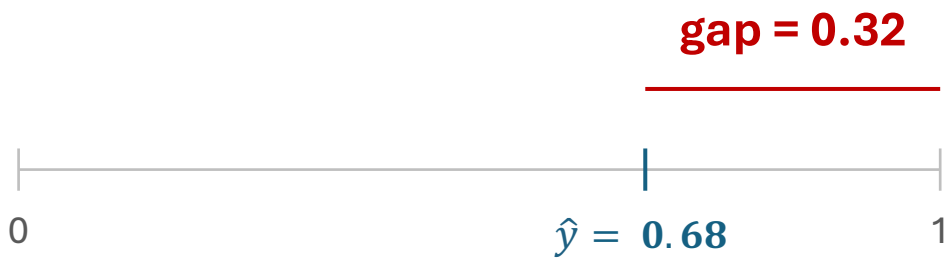
Three Questions

1	How do we turn “wrong” into a number?	**today**
2	Which way do we change a weight to make it smaller?	**today**
3	How do we do that for every weight, affordably?	**next week**

Measure, then find the direction, then reach every layer.

How Wrong Is 0.68?





the network's answer $\hat{y} = 0.68$

the correct answer $y = 1$

$$y - \hat{y} = 1 - 0.68 = 0.32$$

how far off we were, on this one example

The gap between the correct answer and the prediction is where everything starts.

The Sign Problem

Example A

$$y = 1$$
$$\hat{y} = 0.7$$

$$+0.3$$

Example B

$$y = 0$$
$$\hat{y} = 0.3$$

$$-0.3$$

+

their average

0

“no mistakes at all”

Both answers are wrong by the same amount, and the two errors cancelled because one was positive and the other negative.

Plain differences cancel, so two wrong answers can average to no error at all.

Squaring the Error

error	error squared
−0.3	0.09
−0.1	0.01
0.3	0.09
0.9	**0.81**

9 times the error → 81 times the recorded value

$$(y - \hat{y})^2$$

Two reasons for squaring

- Large mistakes count for much more
- It gives a smooth curve, which we return to later

Squaring removes the sign and makes large mistakes count for much more.

Mean Squared Error

$$L = \frac{1}{N}\sum(y - \hat{y})^2$$

add the squared error of every example, then divide by the number of examples N

y	$\hat{y}$	difference	squared
1	0.68	0.32	0.1024
0	0.30	−0.30	0.0900
1	0.90	0.10	0.0100
0	0.20	−0.20	0.0400

total 0.2424

$\boldsymbol{L = 0.0606}$

mean squared error

One number now stands for the performance of the network on the whole dataset.

Not Every Task Uses This Loss

Predicting a quantity

a temperature, a concentration, a size

→ **squared error**

Choosing between categories

tumour or not, which of ten digits

→ **a different loss, later in the course**

Both are built on the same idea: one number that grows when the network is wrong.

The loss is chosen to fit the task, and we will meet others later.

The Loss Depends on the Weights

FIXED

given to us, we cannot touch them

- the inputs
- the labels

FREE

the only things we can change

the nine numbers

six weights and three biases

$$L \text{ depends on } w_1, w_2, \ldots, w_9$$

The data cannot change, so the loss is a function of the weights alone.

Try a Small Change and See

The simplest thing anyone could try

1. Start from any values
2. Run the network, compute the loss
3. Change one number a little, compute it again
4. If the loss went down, that was the right way

Doing it, on one weight

from the second middle neuron to the output

$$w = 0.50 \qquad L = 0.1029$$

↓ nudge it by 0.01

$$w = 0.51 \qquad L = 0.0981$$

The loss went down, so this weight should increase.

Change one weight slightly and watch what the loss does.

The Number That Matters

the weight moved by

+ 0.01

the loss moved by

– 0.0048

$$\frac{-0.0048}{0.01} = -0.48$$

the slope

of the loss with respect to this weight

Increase this weight by one whole unit and the loss goes down by about 0.48.

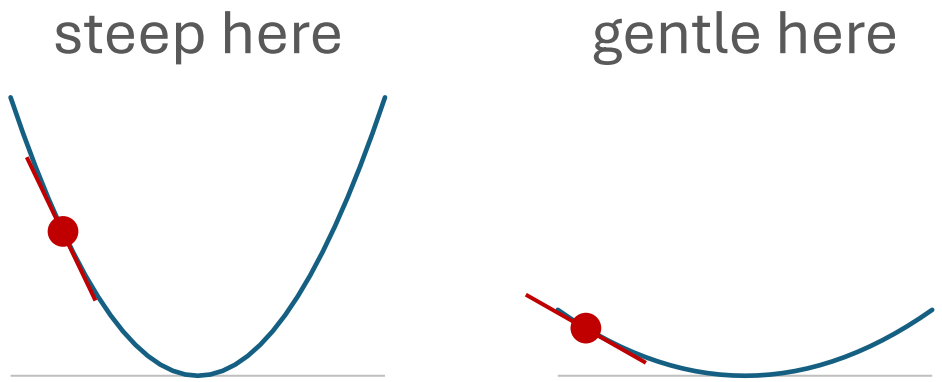
The slope is not fixed. It describes the situation at the value the weight has right now.

How much the loss changes per unit of weight change is the one number we need.

Direction and Steepness

slope	the loss …	so we …
negative	falls as w grows	**increase w**
positive	rises as w grows	**decrease w**

In both cases we move against the sign.



The size says how steep the ground is.

Steep slope, long step. Gentle slope, short step.

The sign says which way to move, and the size says how urgently.

The Update Rule

$$w_{new} = w_{old} - \eta \times slope$$

the old value

the step size

from nudging

Slope negative → the weight grows.
Slope positive → the weight shrinks.

Subtract the slope, scaled, and the weight moves downhill.

Bigger Inputs, Bigger Effect

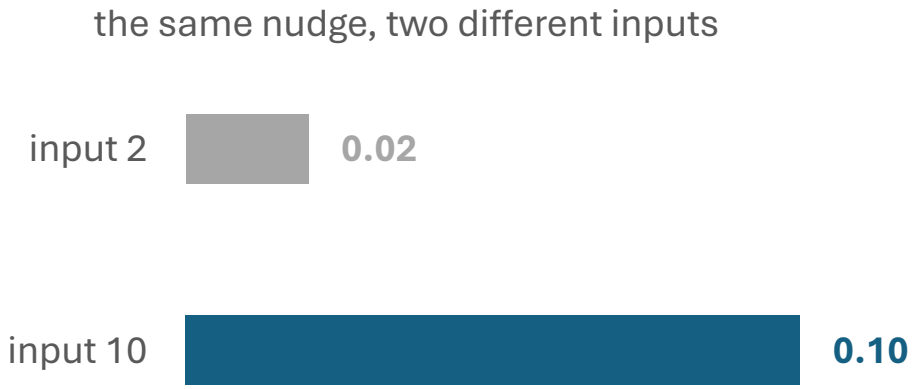
$$z = w_1 x_1 + w_2 x_2 + b$$

one neuron: the weighted sum

Add 0.01 to w_1 — how far does z move?

$x_1 = 2 \longrightarrow$ z moves by 0.02

$x_1 = 10 \longrightarrow$ z moves by 0.10



An input of exactly zero means that weight has no influence at all.

A weight's influence is scaled by the input it is attached to.

How Big a Step?

$$w_{new} = w_{old} - \eta \times slope$$

η is the **learning rate** — how far we walk before measuring again

too small

correct direction, tiny steps

very slow

about right

steady progress downhill

what we want

too large

steps overshoot the bottom

gets worse

The learning rate decides how far one step carries us.

Three Step Sizes, One Problem

one input of value 2 · prediction = $w \times 2$ · correct answer 6 · starting at $w = 1$

$\eta = 0.005$

step	loss
0	16.00
1	14.75
2	13.59
3	12.52
4	11.54

too slow

$\eta = 0.025$

step	loss
0	16.00
1	10.25
2	6.57
3	4.21
4	2.70

steady

$\eta = 0.3$

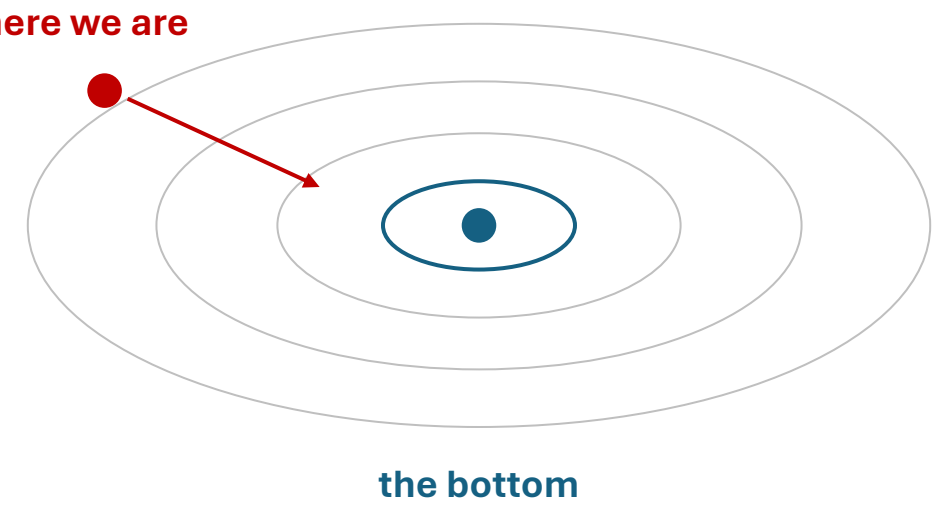
step	loss
0	**16.00**
1	**31.36**
2	**61.47**
3	**120.47**
4	**236.13**

getting worse

Too small wastes time, too large destroys progress
— both are the same mistake about distance.

Two Weights: A Bowl

the loss surface over two weights, seen from above



- Two flat directions: **the two weights**
- Height: **the loss**
- Each weight has its own slope
- The two together point downhill

With nine weights there is no picture
— but the rule for each weight is unchanged.

With two weights the curve becomes a surface, and the idea does not change.

Many Slopes at Once

nudge each one in turn, record its slope

w_1	−0.48
w_2	+0.12
w_3	−1.30
w_4	+0.05
w_5	−0.77
w_6	+0.21
b_1	−0.09
b_2	+0.64
b_3	−0.33

the gradient

the list of all nine slopes

Nothing new here.
The gradient is the numbers we just measured, gathered in one place.

Then update all nine at the same time, each with its own slope.

Follow the gradient, and go down.
gradient descent

The gradient is simply the list of every weight's slope, collected together.

The Whole Method

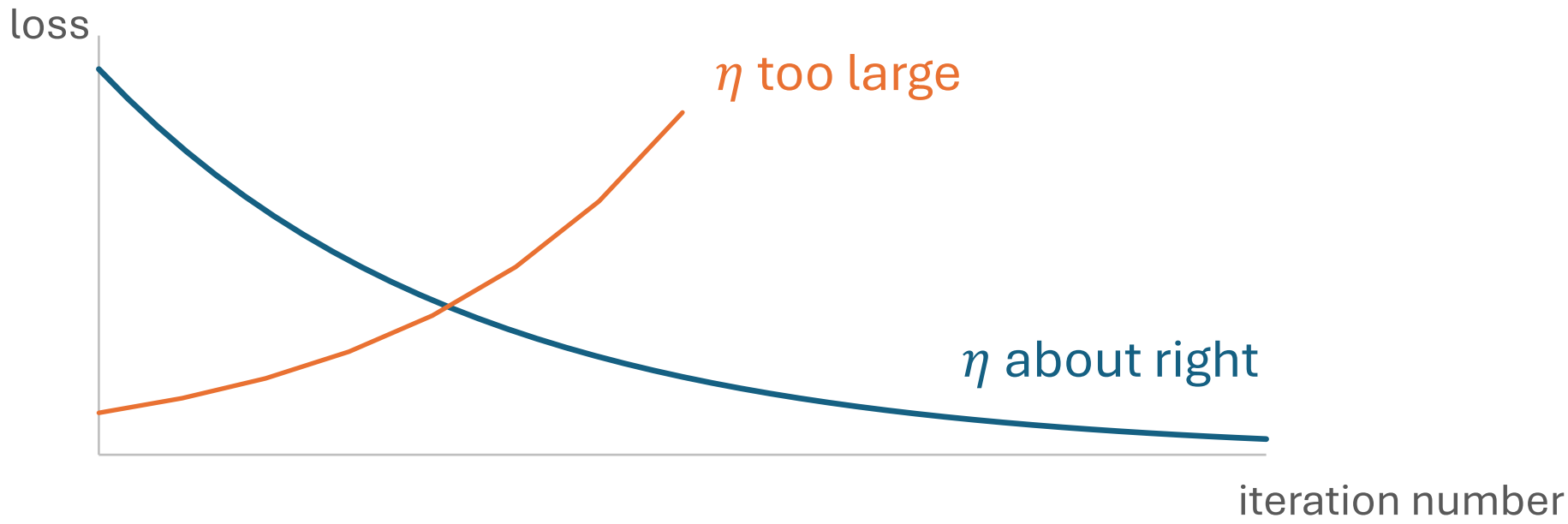
1	**Run the network**	on every training example — forward propagation
2	**Compute the loss**	one number for the whole dataset
3	**Find every slope**	nudge each weight, record how the loss responds
4	**Update every weight**	$w \leftarrow w - \eta \times \text{slope}$

repeat

One pass through these four stages is one **iteration**. 1 iteration uses every example.

Four stages, repeated: predict, measure, find the slopes, move.

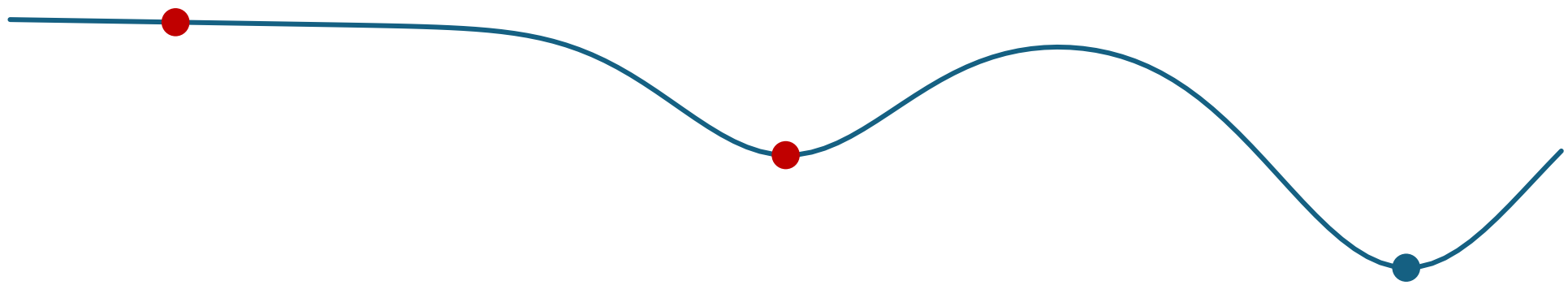
Watching the Loss Fall



Steep at first, then a bend, then a long flat tail — the slopes shrink by themselves as the bottom gets closer.

A falling curve that flattens out is what successful training looks like.

Where the Walk Stops



a long flat stretch
the slope is almost zero

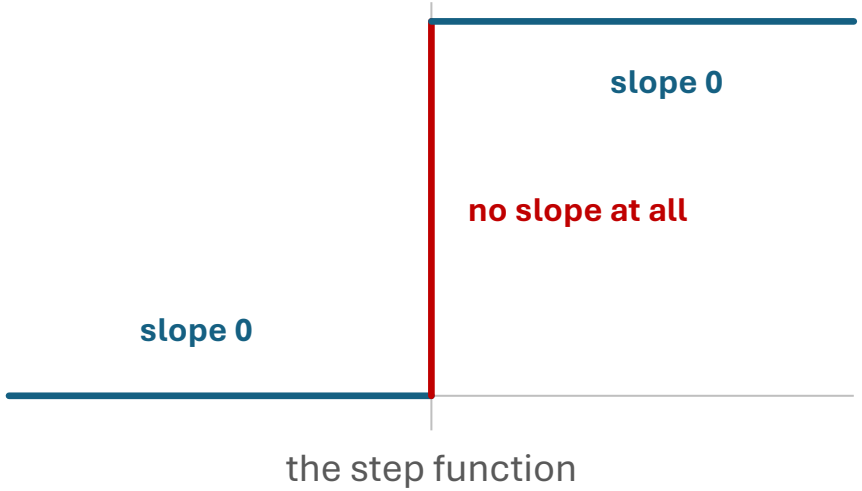
a local minimum
flat, but not the lowest

the true bottom
the best possible here

It walks downhill and stops where the ground is flat.

It finds flat ground, not necessarily the lowest ground.

Why the Step Function Fails



the step function

$$\eta \times 0 = 0$$

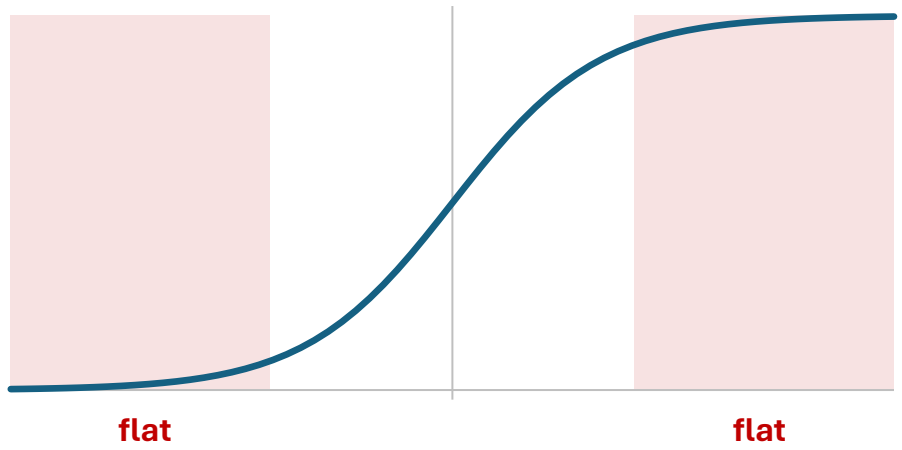
the weight never moves

The update rule produces nothing, at every point.

Also the second reason for squaring: a smooth curve has one slope at every point, a sharp corner does not.

A slope of zero everywhere means the update rule never moves the weight.

Why Flat Ends Slow Learning Down



the sigmoid: smooth, but it saturates at both ends

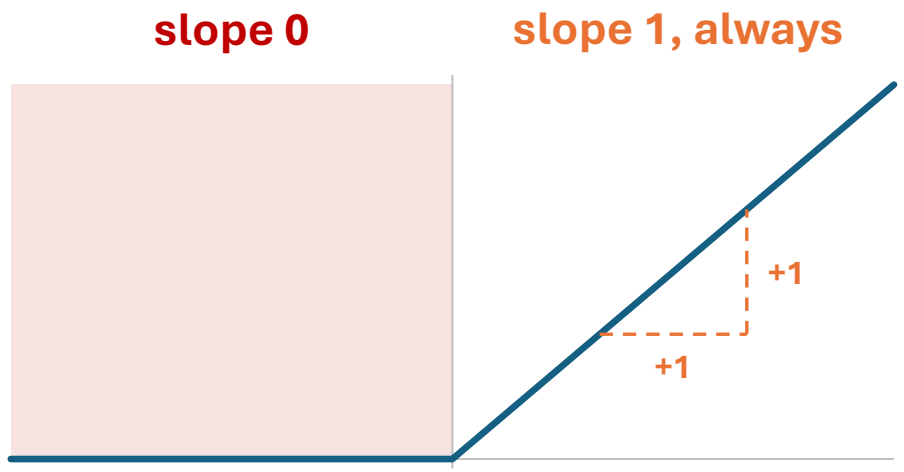
the curve is flat here

- so the output barely changes
- so the loss barely changes
- so the slope is almost zero
- **so the weight barely moves**

Across many layers this accumulates:
the vanishing gradient problem.

Where the curve flattens, the slope shrinks, and the weights attached to it stop moving.

Why ReLU Keeps Learning



negative becomes 0, positive passes through

The line never flattens,
so, the slope never shrinks.

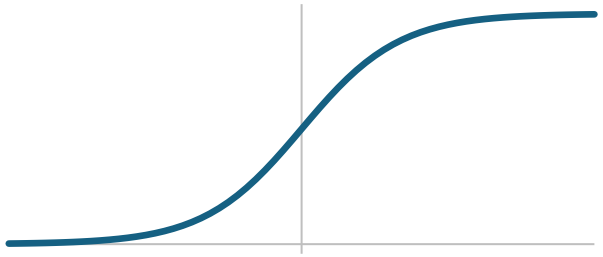
Add 1 to the input and the output rises by exactly 1, whether the input was 3 or 300.

dying ReLU

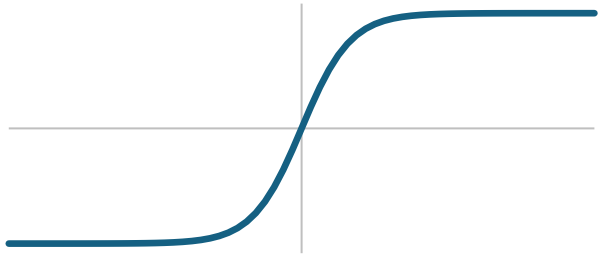
Always negative → output 0 → slope 0 → no signal, and no effect on the loss. There is no way back.

ReLU never flattens on the positive side, so the weights keep receiving a usable slope.

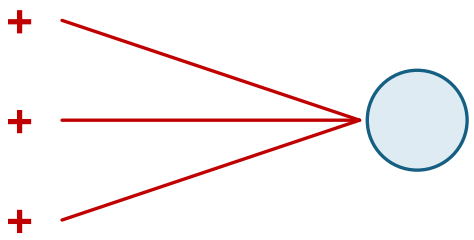
Why Outputs Centred on Zero Help



sigmoid: every output is positive



tanh: outputs on both sides of zero



every input positive

the slopes tend to share a sign

so they tend to move together

If the direct route needs one weight up and another down, that route is not available — the path has to zig-zag.

If every input is positive, the weights tend to move the same way together.

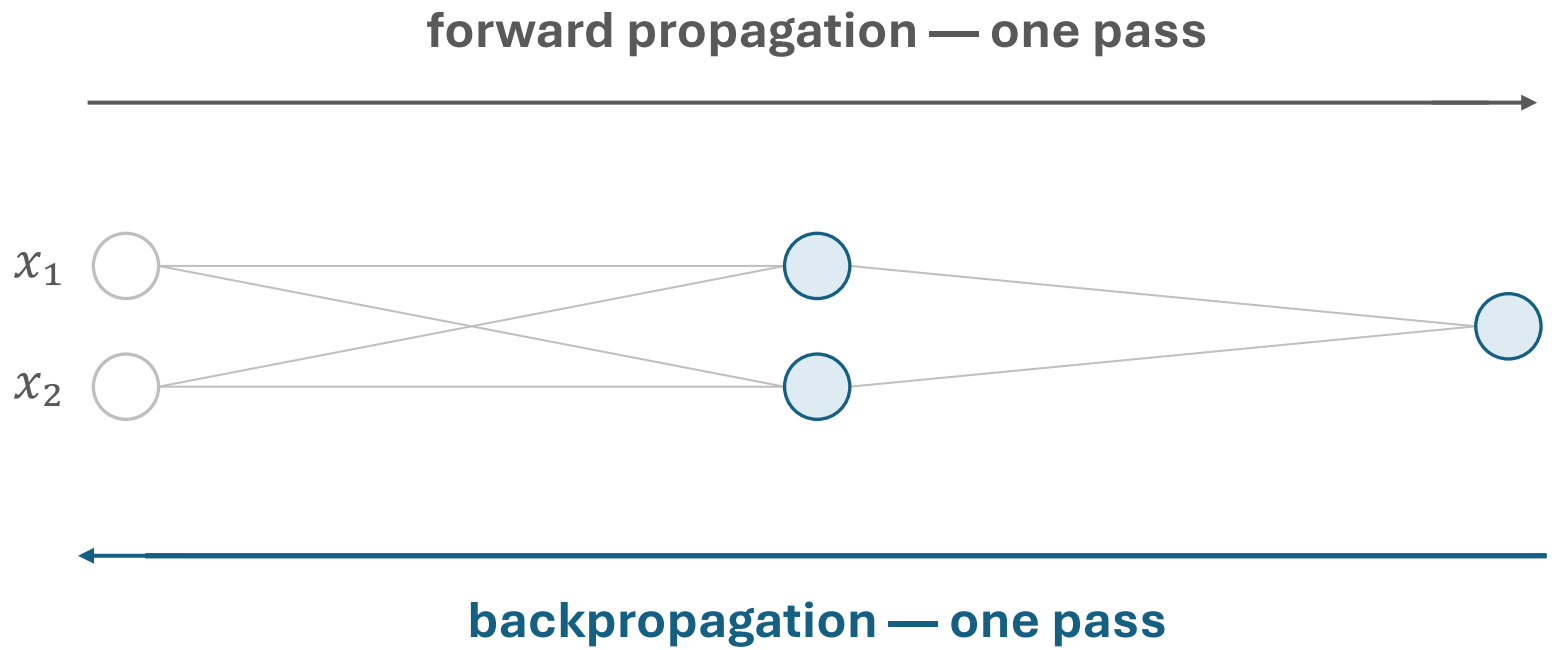
The Cost of Measuring the Slopes

the loss, once	→	1 run over the whole dataset
each weight, nudged	→	1 more run, each
9 weights	→	10 runs — one iteration
100,000 weights	→	100,001 runs — one iteration
thousands of iterations	→	hundreds of millions of runs

And modern networks have billions of weights, not one hundred thousand.

Nudging one weight at a time is correct, and completely impractical.

Next Week: The Way Back



Next week is not what to compute, but how to compute it quickly.

All the slopes can be recovered in a single pass backwards through the network.